Especificación de recursos y DTOs de la API de tutorías

1. Propósito y estado actual

Este documento define los recursos REST previstos para el aplicativo de tutorías y las clases DTO que reciben información del cliente (`Request`) y la que devuelve la API (`Response`).

La implementación actual solo expone `GET /api/estado`, que responde con `EstadoResponse`. Los recursos de estudiantes, tutores, materias y tutorías descritos a continuación son una propuesta; aún no están implementados. Los nombres de campos y reglas se podrán ajustar cuando se acuerden los requisitos del negocio.

2. Convenciones

- Los recursos se nombran en plural y en minúscula: `/estudiantes`, `/tutores`, `/materias`, `/tutorias`.
- Los identificadores se representan como `Long` en Java y como números enteros en JSON.
- Las fechas y horas se envían en formato ISO 8601; por ejemplo, `2026-10-22T14:30:00-05:00`.
- Las operaciones de creación usan `POST`; las de consulta, `GET`; las de actualización completa, `PUT`; y las de eliminación, `DELETE`.
- El cliente no envía campos generados por el servidor, como `id` o marcas de tiempo de auditoría.
- Para solicitudes inválidas se recomienda responder `400 Bad Request`; para un recurso inexistente, `404 Not Found`; para creación exitosa, `201 Created`; para consulta/actualización exitosa, `200 OK`; y para eliminación exitosa sin cuerpo, `204 No Content`.

3. Recursos y operaciones CRUD

3.1 Estudiantes

Representa a las personas que solicitan tutorías.

Método	Ruta	Descripción	Request	Response
GET	`/api/estudiantes`	Lista estudiantes	—	`List<EstudianteResponse>`
GET	`/api/estudiantes/{id}`	Consulta un estudiante	—	`EstudianteResponse`
POST	`/api/estudiantes`	Registra un estudiante	`EstudianteRequest`	`EstudianteResponse` (201)
PUT	`/api/estudiantes/{id}`	Reemplaza los datos editables	`EstudianteRequest`	`EstudianteResponse`
DELETE	`/api/estudiantes/{id}`	Elimina un estudiante	—	Sin cuerpo (204)

3.2 Tutores

Representa a las personas que ofrecen tutorías.

Método	Ruta	Descripción	Request	Response
GET	`/api/tutores`	Lista tutores	—	`List<TutorResponse>`
GET	`/api/tutores/{id}`	Consulta un tutor	—	`TutorResponse`

Método	Ruta	Descripción	Request	Response
POST	`/api/tutores`	Registra un tutor	`TutorRequest`	`TutorResponse` (201)
PUT	`/api/tutores/{id}`	Reemplaza los datos editables	`TutorRequest`	`TutorResponse`
DELETE	`/api/tutores/{id}`	Elimina un tutor	—	Sin cuerpo (204)

3.3 Materias

Catálogo de materias sobre las que se pueden solicitar tutorías.

Método	Ruta	Descripción	Request	Response
GET	`/api/materias`	Lista materias	—	`List<MateriaResponse>`
GET	`/api/materias/{id}`	Consulta una materia	—	`MateriaResponse`
POST	`/api/materias`	Crea una materia	`MateriaRequest`	`MateriaResponse` (201)
PUT	`/api/materias/{id}`	Actualiza una materia	`MateriaRequest`	`MateriaResponse`
DELETE	`/api/materias/{id}`	Elimina una materia	—	Sin cuerpo (204)

3.4 Tutorías

Representa una sesión solicitada o programada entre un estudiante y un tutor para una materia.

Método	Ruta	Descripción	Request	Response
GET	`/api/tutorias`	Lista tutorías	—	`List<TutoriaResponse>`
GET	`/api/tutorias/{id}`	Consulta una tutoría	—	`TutoriaResponse`
POST	`/api/tutorias`	Solicita una tutoría	`TutoriaRequest`	`TutoriaResponse` (201)
PUT	`/api/tutorias/{id}`	Actualiza fecha, duración, modalidad o estado	`TutoriaRequest`	`TutoriaResponse`
DELETE	`/api/tutorias/{id}`	Elimina/cancela una tutoría	—	Sin cuerpo (204)

Para listados, se recomienda agregar paginación y filtros cuando se definan las necesidades de consulta. Por ejemplo: `GET /api/tutorias?estudianteId=12&estado=PROGRAMADA`.

3.5 Estado del servicio (ya implementado)

Método	Ruta	Descripción	Request	Response
GET	`/api/estado`	Informa disponibilidad de la API	—	`EstadoResponse`

Respuesta actual:

```
{
  "servicio": "tutorias-api",
  "estado": "disponible"
}
```

4. Clases Request y Response propuestas

Los DTOs deben ubicarse en `co.edu.autonoma.tutoriasapi.dto`. Se propone separar los objetos de entrada y salida para controlar qué datos acepta y expone la API.

`EstudianteRequest`

```
{
  "nombres": "Ana María",
  "apellidos": "Gómez Ruiz",
  "correo": "ana@example.com",
  "telefono": "3001234567"
}
```

Campos: `nombres` (String, requerido), `apellidos` (String, requerido), `correo` (String, requerido y válido), `telefono` (String, opcional).

`EstudianteResponse`

```
{
  "id": 1,
  "nombres": "Ana María",
  "apellidos": "Gómez Ruiz",
  "correo": "ana@example.com",
  "telefono": "3001234567"
}
```

Incluye `id` generado por el servidor y los datos públicos del estudiante.

`TutorRequest`

```
{
  "nombres": "Carlos",
  "apellidos": "Pérez Díaz",
  "correo": "carlos@example.com",
  "telefono": "3107654321",
  "perfil": "Tutor de matemáticas"
}
```

Campos: `nombres`, `apellidos`, `correo` (requeridos), `telefono` y `perfil` (opcionales). La asignación de materias puede administrarse desde el recurso de tutorías o definirse luego como una relación específica.

`TutorResponse`

Devuelve `id`, `nombres`, `apellidos`, `correo`, `telefono` y `perfil`. No devuelve credenciales ni datos internos.

`MateriaRequest`

```
{
  "nombre": "Cálculo I",
  "descripcion": "Límites, derivadas e integrales"
}
```

Campos: `nombre` (String, requerido) y `descripcion` (String, opcional).

`MateriaResponse`

Devuelve `id`, `nombre` y `descripcion`.

TutoriaRequest

```
{
  "estudianteId": 1,
  "tutorId": 3,
  "materiaId": 2,
  "fechaHora": "2026-10-22T14:30:00-05:00",
  "duracionMinutos": 60,
  "modalidad": "VIRTUAL",
  "observaciones": "Repasar ejercicios para el parcial"
}
```

Campos: `estudianteId`, `tutorId`, `materiaId`, `fechaHora`, `duracionMinutos` y `modalidad` (requeridos); `observaciones` (opcional). Se recomienda que el estado inicial lo asigne el servidor (por ejemplo, `PENDIENTE`) y no confiarlo al cliente al crear.

TutoriaResponse

Devuelve `id`, los identificadores y nombres de estudiante, tutor y materia, `fechaHora`, `duracionMinutos`, `modalidad`, `estado` y `observaciones`. El estado puede usar valores controlados como `PENDIENTE`, `PROGRAMADA`, `COMPLETADA` y `CANCELADA`; el flujo para cambiarlo debe definirse con las reglas del negocio.

5. Organización sugerida en Java

```
co.edu.autonoma.tutoriasapi
■■■ controller
■   ■■■ EstudianteController
■   ■■■ TutorController
■   ■■■ MateriaController
■   ■■■ TutoriaController
■■■ dto
■   ■■■ request
■   ■   ■■■ EstudianteRequest
■   ■   ■■■ TutorRequest
■   ■   ■■■ MateriaRequest
■   ■   ■■■ TutoriaRequest
■   ■■■ response
■       ■■■ EstadoResponse
■       ■■■ EstudianteResponse
■       ■■■ TutorResponse
■       ■■■ MateriaResponse
■       ■■■ TutoriaResponse
■■■ service
```

Estas clases son una propuesta de contrato, no clases ya presentes en el código. Para Spring, pueden implementarse como `record` de Java y anotarse con validaciones (`@NotBlank`, `@Email`, `@NotNull`, `@Min`) en los campos requeridos. La lógica de persistencia y entidades de base de datos queda fuera del alcance de este documento.

6. Decisiones pendientes de negocio

- Si el alta de estudiante/tutor incluye autenticación y manejo de contraseñas.
- Si un tutor puede ofrecer varias materias y cómo se administra esa relación.
- Si `DELETE` elimina definitivamente o realiza una baja lógica; en tutorías podría ser preferible cambiar el estado a `CANCELADA`.
- Si se necesitan endpoints de cambio de estado separados, como `PATCH /api/tutorias/{id}/estado`.
- Reglas de disponibilidad, solapamiento de horarios, duración y modalidad de las sesiones.
- Campos personales adicionales que deban recopilarse o protegerse.